

Discrete Mathematics - Practical 8

Recurrence Relations: worked solutions
with the reasoning spelled out, step by step

Warm-up: what is a recurrence, and what are we asked to do?

A **recurrence relation** defines a sequence by expressing each term through *earlier* terms, together with a few **initial conditions**. For example $a_n = a_{n-1} + a_{n-2}$ with $a_0 = a_1 = 1$ is the Fibonacci rule.

Two very different tasks appear in this practical, and the two problems below are exactly one of each:

- **Solve** a recurrence - find a *closed formula* for a_n that does not refer to earlier terms (Problem 2). The tool is the *characteristic equation*.
- **Build** a recurrence - translate a *counting* question into a recurrence, then run it (Problem 6). The tool is a clever *case split*.

Problem 2 - Solving a linear recurrence

Problem 2

Solve the recurrence relation

$$a_n = 5a_{n-1} - 6a_{n-2}, \quad n > 2,$$

with initial conditions $a_1 = 2$, $a_2 = 10$. ("Solve" = find a closed formula for a_n .)

The idea

First, name the structure. This recurrence is **linear** (each a_i appears to the first power, never a_i^2 or $a_i a_j$), **homogeneous** (nothing is added on the right), and has **constant coefficients** (5 and -6 never change with n). For exactly this kind of recurrence the right guess is a geometric sequence $a_n = r^n$. It is *not* pulled from a hat; three observations force it.

(i) **The simplest recurrence is already geometric.** A one-step rule $a_n = b a_{n-1}$ multiplies by the same constant b at every step, so

$$a_n = b a_{n-1} = b^2 a_{n-2} = \dots = a_0 b^n.$$

A fixed multiplier applied over and over *is* geometric growth. Our two-step rule is just two such growths blended together, so we expect geometric pieces.

(ii) **Geometric is the one shape where stepping back means scaling by a constant.** If $a_n = r^n$, then $a_{n-1} = \frac{1}{r} a_n$ and $a_{n-2} = \frac{1}{r^2} a_n$. A constant-coefficient recurrence does precisely this: it ties a term to its earlier neighbours through fixed numbers. The geometric sequence is built to fit that move.

(iii) **It collapses infinitely many equations into one.** The recurrence must hold for *every* n at once, which is infinitely many demands. Feed in $a_n = r^n$ and every term shares the factor r^{n-2} ; cancel it and the index n disappears entirely, leaving the single equation $r^2 = 5r - 6$. So one well-chosen number r satisfies the rule at step 3, step 4, step 100, and

onward, simultaneously. *That* is the real reason the guess works: the geometric shape is the only one that makes a constant-coefficient recurrence say the very same thing at every step. The values of r that fit are the growth rates hidden inside the recurrence.

Step 1 - Substitute the guess. Put $a_n = r^n$ into $a_n = 5a_{n-1} - 6a_{n-2}$:

$$r^n = 5r^{n-1} - 6r^{n-2}.$$

Divide by the smallest power r^{n-2} (allowed since $r = 0$ only gives the trivial sequence):

$$r^2 = 5r - 6 \iff \boxed{r^2 - 5r + 6 = 0}.$$

This is the **characteristic equation**.

Step 2 - Solve it.

$$r^2 - 5r + 6 = (r - 2)(r - 3) = 0 \implies r_1 = 2, \quad r_2 = 3.$$

We found *two* different geometric solutions: 2^n and 3^n .

Step 3 - Combine them (superposition). Because the recurrence is linear and homogeneous, *any* combination of solutions is again a solution. With two distinct roots the general solution is

$$a_n = C \cdot 2^n + D \cdot 3^n,$$

where C, D are constants. We have exactly two unknowns to match our two initial conditions, and that is no coincidence.

Why is $C \cdot 2^n + D \cdot 3^n$ *every* solution, not just some of them? Suppose another sequence also obeys $a_n = 5a_{n-1} - 6a_{n-2}$ and happens to agree with our formula at two consecutive indices. Then they must agree *everywhere*: each next term is forced by the previous two (and the rule runs backwards as well, fixing the earlier terms). So a solution is pinned down completely by any two consecutive values, and the constants C, D give exactly enough freedom to match a_1 and a_2 . Once they are set, the formula *is* the sequence.

Step 4 - Use the initial conditions.

$$\begin{cases} a_1 = 2 : & 2C + 3D = 2, \\ a_2 = 10 : & 4C + 9D = 10. \end{cases}$$

Multiply the first equation by 2 and subtract from the second:

$$(4C + 9D) - (4C + 6D) = 10 - 4 \implies 3D = 6 \implies D = 2,$$

and then $2C + 3(2) = 2 \implies C = -2$.

Step 5 - Write the closed formula.

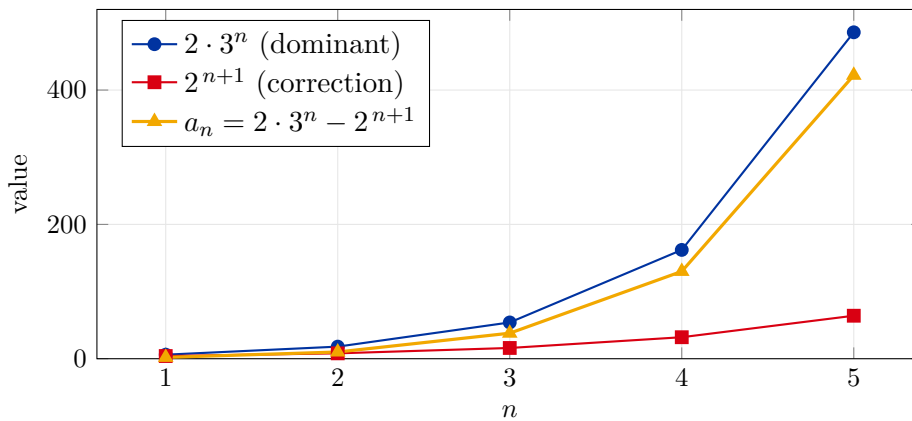
$$a_n = -2 \cdot 2^n + 2 \cdot 3^n = 2 \cdot 3^n - 2^{n+1}.$$

Step 6 - Always check.

$$\begin{aligned} a_1 &= 2 \cdot 3 - 2^2 = 6 - 4 = 2 \checkmark & a_2 &= 2 \cdot 9 - 2^3 = 18 - 8 = 10 \checkmark \\ a_3 &\stackrel{\text{formula}}{=} 2 \cdot 27 - 2^4 = 38 & \text{vs} & \quad a_3 \stackrel{\text{recurrence}}{=} 5 \cdot 10 - 6 \cdot 2 = 38 \checkmark \end{aligned}$$

$$\textbf{Answer: } a_n = 2 \cdot 3^n - 2^{n+1}$$

Seeing the formula. The sequence is the difference of two geometric "engines". The 3^n engine (blue) wins the race; the 2^{n+1} engine (red) is a shrinking correction, so a_n (orange) hugs $2 \cdot 3^n$ more and more closely as n grows.



n	1	2	3	4	5	6
a_n	2	10	38	130	422	1330
a_n/a_{n-1}		5	3.80	3.42	3.25	3.15

Look at the bottom row: the step-to-step growth rate a_n/a_{n-1} is settling toward 3, the larger characteristic root. For large n the 3^n term dwarfs the 2^{n+1} term, so the sequence ends up growing almost exactly like 3^n . This is the geometric guess paying off in reverse: the roots we solved for really are the growth rates the recurrence was hiding all along.

★ Method to remember

For a second-order linear homogeneous recurrence $a_n = p a_{n-1} + q a_{n-2}$:

1. Write the characteristic equation $r^2 = pr + q$.
2. **Two distinct roots** $r_1 \neq r_2$: general solution $a_n = C r_1^n + D r_2^n$.
3. **Repeated root** $r_1 = r_2 = r$: general solution $a_n = (C + Dn) r^n$ (an extra factor n - needed because r^n alone gives only one solution).
4. Find C, D from the two initial values, then verify.

Problem 6 - Building a recurrence to count

Problem 6

How many binary strings (strings of 0s and 1s) of length 10 contain **two consecutive zeros** (i.e. the block "00" appears somewhere)?

The idea

Counting strings that *do* contain "00" directly is painful: the block can sit in many overlapping places, and you would over-count strings that contain it more than once.

Complementary counting. It is far easier to count the *opposite* - strings with **no** "00" - and subtract from the total:

$$\#(\text{contain } 00) = \underbrace{2^{10}}_{\text{all strings}} - \#(\text{avoid } 00).$$

And "avoid 00" is exactly the kind of property that builds up nicely one symbol at a time - so it satisfies a recurrence.

Step 1 - Set up the count we actually want. Let

$$f(n) = \#\{\text{binary strings of length } n \text{ with no two adjacent 0s}\}.$$

Step 2 - Find a recurrence by looking at the *last* symbol. Take any "good" string of length n and look at how it ends. There are exactly two mutually exclusive cases, and each case is glued onto a shorter good string:

ends in 1: good string of length $n - 1$ 1 $\rightarrow f(n - 1)$ ways

ends in 0: good string, length $n - 2$ 1 0 $\rightarrow f(n - 2)$ ways

the symbol before a final 0 must be 1, otherwise we'd create "00"

- **Ends in 1:** whatever comes before can be *any* good string of length $n - 1$ (adding a 1 never creates "00"). That gives $f(n - 1)$ strings.
- **Ends in 0:** to avoid "00", the symbol just before must be 1, so the string ends in "10". Whatever comes before is any good string of length $n - 2$. That gives $f(n - 2)$ strings.

The two cases never overlap and cover everything, so

$$f(n) = f(n - 1) + f(n - 2).$$

This is the **Fibonacci rule** - counting reveals it on its own!

Step 3 - Base cases (count tiny strings by hand).

$$\begin{aligned} n = 1 : \{0, 1\} &\Rightarrow f(1) = 2, \\ n = 2 : \{01, 10, 11\} \text{ (not 00)} &\Rightarrow f(2) = 3, \\ n = 3 : \{010, 011, 101, 110, 111\} &\Rightarrow f(3) = 5. \end{aligned}$$

(Notice $5 = 3 + 2$, confirming the recurrence.)

Step 4 - Run the recurrence up to $n = 10$.

n	1	2	3	4	5	6	7	8	9	10
$f(n)$	2	3	5	8	13	21	34	55	89	144

So there are $f(10) = 144$ length-10 strings that *avoid* "00".

Step 5 - Complementary count.

$$\#(\text{contain 00}) = 2^{10} - f(10) = 1024 - 144 = 880.$$

Answer: 880 binary strings of length 10 contain two consecutive zeros.

★ **Method to remember**

Two transferable tricks here:

1. *Complementary counting* - when "at least one bad pattern" is messy, count the clean strings and subtract from the total.

2. *Last-symbol case split* - to build a recurrence for strings with a local forbidden pattern, branch on how the string ends; each branch attaches a fixed suffix to a shorter valid string.